Notes (Week 7 Wednesday)

These are the notes I wrote during the lecture.

```
Transition systems:
   a set of states,
   a transition relation describing how you can
   move between states

A *run* of a transition system:
   a sequence of states you can visit by following
   transitions

A *maximal* run is a run that cannot
  be extended.

Some (but not all) important info about a
  transition system is captured by its set of runs

What sort of things might  you ask about a transition system?

 What *properties* does it have.
 Where a property of a transition system is
 a set of runs that includes all the runs
 of that transition system.

   Property: "Termination"
             read: the set of all terminating runs

   There are two kinds of properties:
   - safety properties "something bad never happens"
   - liveness properties "something good will happen"
   Neat result (Alpern & Schneider):
     Every property is the intersection of a
       safety property and a liveness property

   The program will allocate exactly 100MB
   this is the intersection of:
   - the program will allocate at least 100MB (liveness)
   - the program will allocate at most 100MB (safety)

   Safety and liveness properties have
    completely different proof techniques

   Find an *invariant*:
```

```
    Something that is true initially,
    stays true after every transition,
    and is false when the "bad thing" happens


  Preconditions: what's true initially
  Postconditions: what we want to be true finally

  Total correctness =
    if the precondition is true initially,
    the program (transition system) *will*
    reach a final state that satisfies the
    postcondition

  Total correctness =
    partial correctness (safety)
    + termination    (liveness)

  Partial correctness =
    if the precondition is true initially,
    then *if* the program (transition system)
    terminates, the final state satisfies the
    postcondition

Once we have a *program semantics*, we'll have
a systematic way of extracting such transition
systems from program text in a principled way.
For now, we're winging it.

 ψ ≡ rx^y = m^n

 if y is even and
  ψ(x,y,r)
 then
  ψ(x^2,y/2r)


 if y is even and
  rx^y = m^n
 then
  r(x^2)^(y/2) = m^n


To show termination for exponentiation
 we need to find a measure

 f : ℕ × ℕ × ℕ → ℕ
 f(x,y,r) = y
 Show: whenever p → q, f(p) > f(q)
```

```
You may notice a connection
 to well-founded orders

A measure is a special case of a
 well-founded order

A wfo is a poset (S,<)
 with no infinitely descending chains.

A measure gives you a wfo:
  if f is a measure on S, then
    (S,<) is a poset
  where
    x < y   iff f(x) < f(y)

In *most* cases, finding a measure works.
Some transition systems terminate
despite having no measure.

Convention:
  An arrow labelled a,b where a,b ∈ Σ
  Is shorthand for two arrows.
  One labelled a and one labelled b.

The *language* of an automaton
 is the set of words that cause
 the automaton to terminate an
 accepting state

 10   is not in the language

 L_A(q) = the set of words that takes us
          from q to a final state

If you start from a DFA A,
 and construct A' by inverting the
 set of final states

then L(A') = L(A)^C

 One state: q

   δ(q,a) = q
 final states: {}

x means "either this symbol was an A,
         or index-out-of-bounds"
```

```
In a DFA, a word w is accepted if
 by consuming w, we *must* reach
 an accepted state.
In an NFA, a word w is accepted if
 by consuming w, we *may* reach
 an accepted state.

Execute an automaton by
 tracking not which state we're in
 but, which set of states we *could* be in

NFA:s don't add any expressive power.
```